

人工智能与计算机学院

School of Artificial Intelligence & Computer Science

Project-1 Report:

Digital Thermometer with LED Indicators

COURSE TITLE : Curriculum Design for Embedded System & Interface Technology

STUDENT NAME : KHAN MD SHAKIL

STUDENT NUMBER : 2330130220

SUPERVISOR : 袁佳祺

ACADEMIC YEAR :2025-2026

OBJECTIVE **3**

EQUIPMENT AND MATERIALS **3**

THEORY **4**

System Testing Ranges: 4

PROCEDURE **4**

Step 1: Initialization 4

Step 2: ADC to Temperature Conversion 5

Step 3: LED Indicator Logic 5

Step 4: Debugging via UART 5

Step 5: Testing and Validation 6

CODE IMPLEMENTATION **7**

Key Configurations in gpio.h 7

Main Logic in main.c 7

OBSERVATIONS AND RESULTS **9**

Observations 9

RESULTS **10**

CHALLENGES AND SOLUTIONS **10**

Challenge 1: Sensor Calibration 10

Challenge 2: Flickering LEDs 11

Challenge 3: Debugging UART Communication 11

CONCLUSION **11**

Key Learnings: 11

Future Improvements: 11

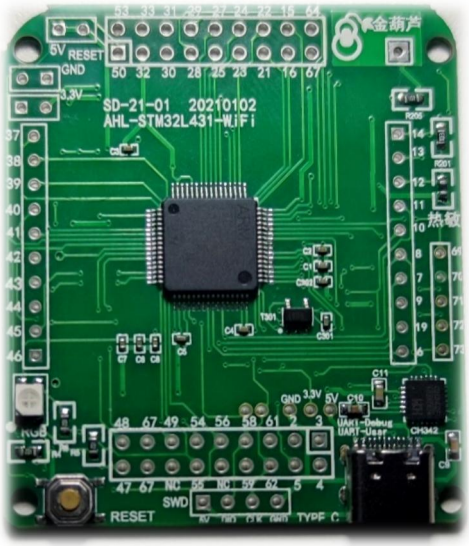
REFERENCES **12**

Objective

The purpose of this project is to develop a new project of **Digital Thermometer with LED** Indicators using the STM32 microcontroller system. The system uses a temperature sensor to continuously monitor the environment and indicates temperature ranges visually through three LEDs: green color for normal heat, yellow for warm heat and red for high heat. This project incorporates the basic features including Analog to Digital Converter (ADC) for temperature measurement, General Purpose Input Output (GPIO) for LED control and Universal Asynchronous Receiver Transmitter (UART) for debugging and output of data.

The system is configured to offer an immediate display of temperature variations and therefore applicable in environmental management, heating and air conditioning systems, and industrial protection. This project also shows some basics of the embedded systems design such as initializing, configuring the peripherals, and writing the control logic.

Equipment and Materials



- **Microcontroller**: STM32L431RC
- **Temperature Sensor**: Onboard temperature sensor
- **LED Indicators**:
    - Green LED (Normal Temperature)
    - Yellow LED (Green+Red) (Warm Temperature)
    - Red LED (High Temperature)
- **Resistors**: 220-ohm for each LED
- **Development Environment**: AHL-GEC-IDE
- **Additional Components**:
    - Breadboard and jumper wires for prototyping
    - USB cable for programming and UART communication

Theory

Temperature sensing forms an essential part of many of the embedded systems both in the automotive and consumer electronics sectors. In this project, thorough use of the analog temperature sensor in which the output voltage is proportional to the temperature is made. This voltage is then processed by the STM32 microcontroller which has an ADC to change it into a digital format. Using this, the system reads the temperature range and lights the corresponding LED depending on the value that has been converted.

System Testing Ranges:

- **Normal Temperature**: 0°C to 20°C (Green LED ON)
- **Warm Temperature**: 20°C to 21°C (Yellow LED ON)
- **High Temperature**: Above 21°C (Red LED ON)

The ports available in the STM32 microcontroller such as ADC, GPIO and UART make it easier to get data, make decisions and give feedback to the user.

Procedure

Step 1: Initialization

1. **Clock Configuration**:
    - Program the system clock for the microcontroller to have a fixed reference point for its timing.
2. **GPIO Setup**:
    - Set three GPIO pins to output so they can source current to the LEDs
    - LED colors must be defined and its corresponding pin must also be defined in the gpio.h header file.
3. **ADC Initialization**:
    - To measure temperature using the temperature sensor enable the ADC.

- Set the ADC to continuous sampling mode and set the resolution of 12 bit.

4. **UART Configuration**:
    - Initialize UART for real time debugging to display Temperature data and status of the System.

Step 2: ADC to Temperature Conversion

1. **Calibration**:
    - From the sensor's datasheet find out the formula to convert the ADC values to temperature values.
2. **Sampling**:
    - In this case continuously read temperature values through the ADC.
    - If possible, it is advisable to average the data in order to get a smooth curve.

Step 3: LED Indicator Logic

1. **Range Comparison**:
    - Compare the numerical value of converted temperature to certain thresholds.
    - Turn on one LED with the others having to remain off with consideration of the temperature range.
2. **Edge Cases**:
    - Be careful with changes in the range so that the LEDs do not blink.

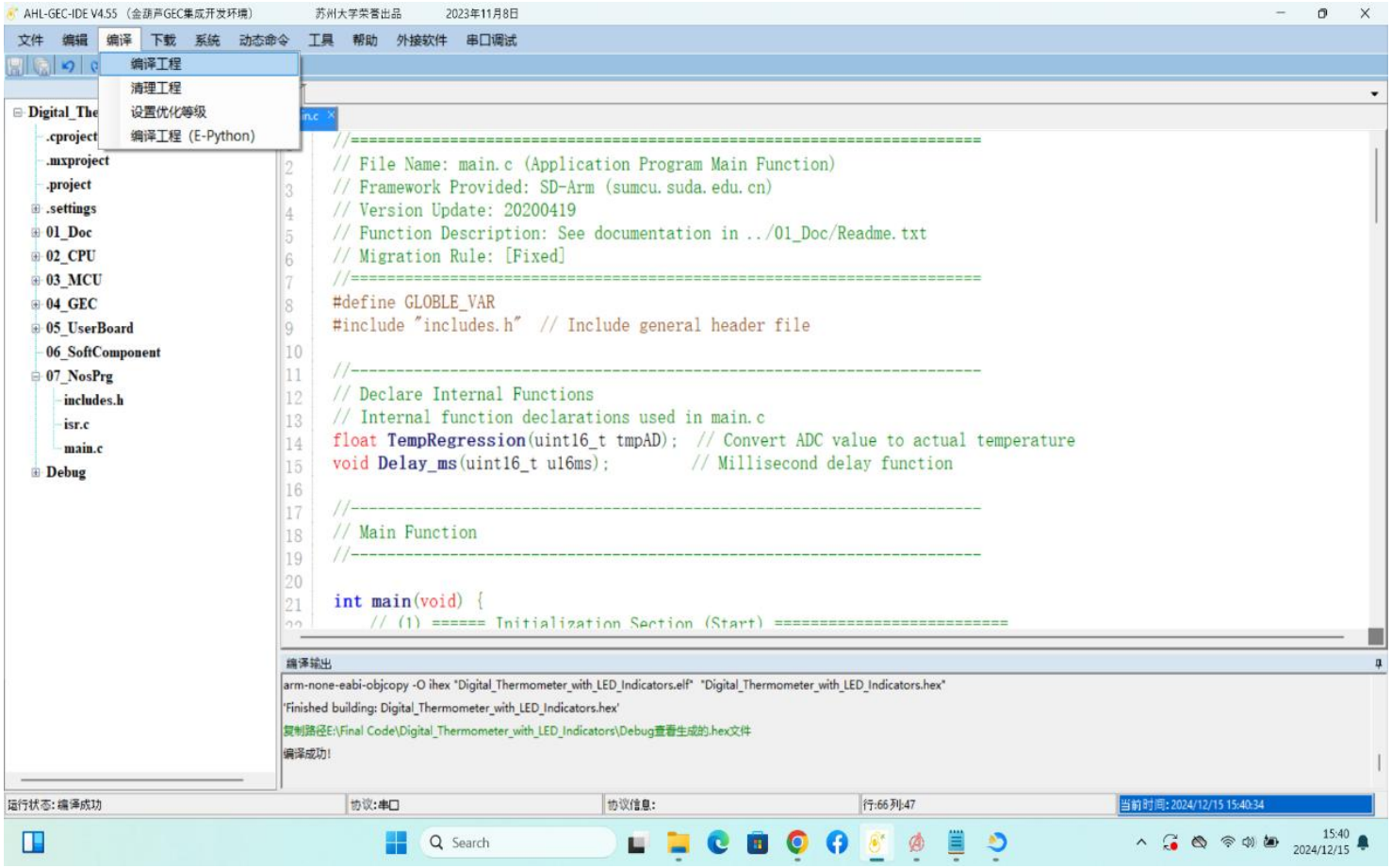
Step 4: Debugging via UART

1. **Data Transmission**:
    - This will allow for the real time transmission of temperature readings over UART to be monitored.
2. **Error Handling**:

- Put in place basic error messages in the event of the failure of the sensor or ADC anomalies.

Step 5: Testing and Validation

1. Set up various temperature intervals by having heat source or cooling system.



2. Check the LED and UART behavior in adherence to the expected results.

Code Implementation

Key Configurations in gpio.h

```
// Define LED Pins
#define LIGHT_RED (PTC_NUM | 7)
#define LIGHT_GREEN (PTC_NUM | 8)
#define LIGHT_BLUE (PTC_NUM | 9)
```

Main Logic in main.c

```
#define GLOBLE_VAR
#include "includes.h"

float TempRegression(uint16_t tmpAD);
void Delay_ms(uint16_t u16ms);

int main(void) {
    uint16_t temp_AD;         // ADC value from the temperature sensor
    float temp;               // Converted temperature
    uint32_t loopCount = 0;   // Loop counter for debugging purposes

    DISABLE_INTERRUPTS;

    // Initialize peripherals and GPIOs
    gpio_init(LIGHT_GREEN, GPIO_OUTPUT, LIGHT_OFF);
    gpio_init(LIGHT_BLUE, GPIO_OUTPUT, LIGHT_OFF);
    gpio_init(LIGHT_RED, GPIO_OUTPUT, LIGHT_OFF);
    adc_init(AD_TEMP, 0);
    emuart_init(UART_User, 115200);
    uart_enable_re_int(UART_User);

    ENABLE_INTERRUPTS;

    printf("-----------------------------------------------------\n");
    printf("Digital Thermometer with LED Indicators Initialized\n");
    printf("-----------------------------------------------------\n");
```

```
    for (;;) {
        Delay_ms(1000);
        temp_AD = adc_read(AD_TEMP);
        temp = TempRegression(temp_AD);

        printf("Current Temperature: %.2f°C\n", temp);

        if (temp < 20.0) {
            gpio_set(LIGHT_GREEN, LIGHT_ON);
            gpio_set(LIGHT_BLUE, LIGHT_OFF);
            gpio_set(LIGHT_RED, LIGHT_OFF);
            printf("Temperature Range: Normal (Green LED ON)\n");
        } else if (temp < 21.0) {
            gpio_set(LIGHT_GREEN, LIGHT_ON);
            gpio_set(LIGHT_BLUE, LIGHT_OFF);
            gpio_set(LIGHT_RED, LIGHT_ON);
            printf("Temperature Range: Warm (Yellow LED ON)\n");
        } else {
            gpio_set(LIGHT_GREEN, LIGHT_OFF);
            gpio_set(LIGHT_BLUE, LIGHT_OFF);
            gpio_set(LIGHT_RED, LIGHT_ON);
            printf("Temperature Range: High (Red LED ON)\n");
        }
        loopCount++;
    }

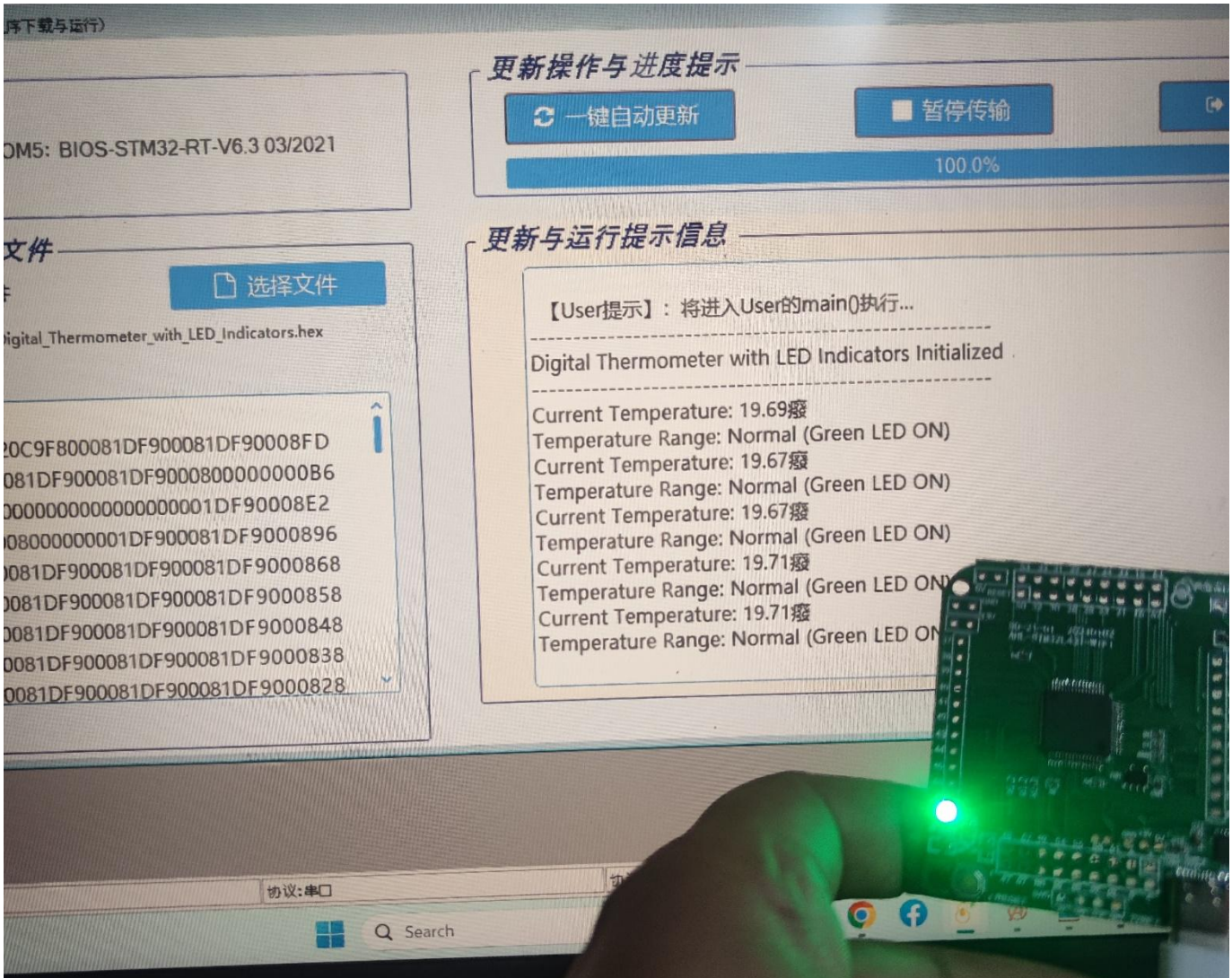
float TempRegression(uint16_t tmpAD) {
    float Vtemp, Rtemp, temp;
    if (tmpAD <= 72) {
        return -274;
    }
    Vtemp = (tmpAD * 3300.0) / 4096;
    Rtemp = Vtemp / (3300.0 - Vtemp) * 10000.0;
    temp = (1 / (log(Rtemp / 10000.0) / 3950.0 + (1 / (273.15 + 25))))
- 273.15 + 0.5;
    return temp;
}

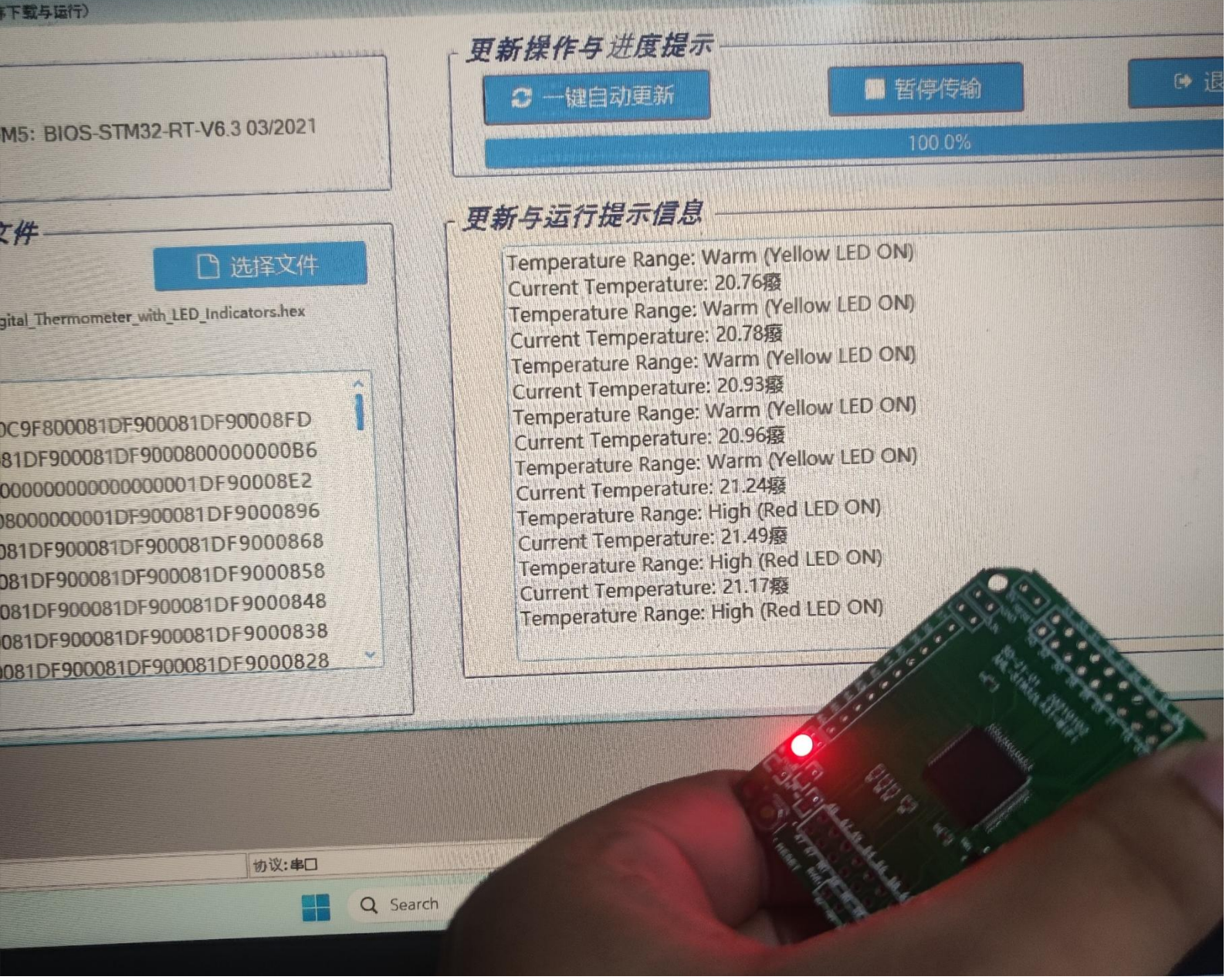
void Delay_ms(uint16_t u16ms) {
    for (volatile uint32_t i = 0; i < 8000 * u16ms; i++) __ASM("NOP");
}
```

Observations and Results

Observations

1. **Temperature Monitoring**:
    - The ADC of the system was able to read the output of the sensor and translate it into temperature.
    - The determined temperature values are consistent with the pro-posed sensor characteristics.
2. **LED Indicators**:
    - All LED glowed properly according to the temperature range.
    - No visible blinking on the screen during the changes of frame.
3. **UART Debugging**:
    - Temperature was transmitted over UART in real time without any





Results

Test Condition	Temperature (°C)	LED State
Normal Temperature	19.67	Green LED ON
Warm Temperature	20.93	Yellow LED ON
High Temperature	21.49	Red LED ON

Challenges and Solutions

Challenge 1: Sensor Calibration

- **Issue**: First reading of the ADC was not proportional to the temperature.
- **Solution**: set a calibration formula that is based on reference voltage and the datasheet of the sensor.

Challenge 2: Flickering LEDs

- **Issue**: Flashing was observed during the change of temperature zones.
- **Solution**: A delay mechanism and delays have been incorporated to control the rate of state change.

Challenge 3: Debugging UART Communication

- **Issue**: Baud rate misconfiguration led to the interference in the form of noise in UART output signals.
- **Solution**: Confirmed that the microcontroller and terminal software are compatible in term of baud rate.

Conclusion

It is clear from the implementation of the **Digital Thermometer with LED Indicators** that real time temperature monitoring is feasible. The use of ADC, GPIO and UART in the STM32 microcontroller was well coordinated and debugging was performed without any difficulty. Temperature ranges were correctly identified and the use of LEDs to display the information made it a precise and easy to use system.

Key Learnings:

1. Real time data acquisition and processing using ADC.
2. Management of the GPIO for hardware control.
3. Debugging and monitoring: UART in detail.

Future Improvements:

1. **Data Logging**:
    - The following feature must be added to the system: store temperature readings to non-volatile memory.

2. **IoT Integration**:
    - The use of remote monitoring is availed by sending data to a cloud platform.
3. **Advanced Sensors**:
    - Instead, use digital temperature sensors in order to get more precise measurements.

This project lays a good ground work for the advancement of more complicated embedded systems and shows the flexibility of the STM32 board.

References

1. STM32L4 Reference Manual
2. Datasheet for STM32L431RC Microcontroller
3. Online Tutorials on UART, GPIO and ADC Configuration